## 011 BE Practical

## Hyperledger Fabric – Comparative Exploration

CODE:

```
<!DOCTYPE html>

<html lang="en" class="scroll-smooth">

<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Hyperledger Fabric vs. Ethereum - Comparative Exploration</title>


   <!-- Tailwind CSS -->

   <script src="https://cdn.tailwindcss.com"></script>


   <!-- Chart.js -->

   <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>


   <!-- Chosen Palette: Warm Minimalist Enterprise (Stone/Slate/Teal) -->

   <!-- Application Structure Plan: The SPA is structured as an interactive project dashboard. It begins with the high-level objective (Hero), followed by interactive environment prerequisites (Requirements section). The core analysis is presented via a visual Comparison Dashboard using a Radar chart and interactive tabs to contrast Fabric and Ethereum. Finally, the Network Architecture section uses HTML/CSS layout to map out the physical deployment flow without relying on external diagramming tools. This structure logically guides the user from setup to conceptual comparison to architectural implementation. -->

   <!-- Visualization & Content Choices:

       1. Objective -> Text Hero Block -> Immediate context.

       2. Requirements -> Interactive Checklist -> Engagement, allows users to track setup readiness.

       3. Fabric vs Ethereum metrics -> Chart.js Radar Chart -> Ideal for multi-variable comparative analysis (Performance, Privacy, etc.). NO SVG used.

       4. Detailed Comparison -> Interactive Tabs -> Keeps UI clean, prevents information overload while allowing deep dives.
```

5. Architecture -> CSS Flexbox Flowchart -> Visually represents the Orderer/Peer/Chaincode relationship using structured HTML and Unicode arrows. NO Mermaid JS or SVG used.

```
-->
<!-- CONFIRMATION: NO SVG graphics used. NO Mermaid JS used. -->


<script>
  tailwind.config = {
    theme: {
      extend: {
        colors: {
          'brand-stone': '#f5f5f4',
          'brand-slate': '#1e293b',
          'brand-teal': '#0f766e',
          'brand-accent': '#ccfbf1'
        },
        fontFamily: {
          sans: ['Inter', 'system-ui', 'sans-serif'],
        }
      }
    }
  }
</script>


<style>
  /* Essential Chart Container Constraints */
  .chart-container {
    position: relative;
    width: 100%;
    max-width: 600px;
    margin-left: auto;
    margin-right: auto;
```

```
        height: 350px;
        max-height: 450px;
    }
    @media (min-width: 768px) {
        .chart-container {
            height: 450px;
        }
    }

    /* Custom Scrollbar */
    ::-webkit-scrollbar { width: 8px; }
    ::-webkit-scrollbar-track { background: #f1f1f1; }
    ::-webkit-scrollbar-thumb { background: #cbd5e1; border-radius: 4px; }
    ::-webkit-scrollbar-thumb:hover { background: #94a3b8; }

    /* Checklist animation */
    .req-item.checked {
        text-decoration: line-through;
        color: #94a3b8;
        transition: all 0.3s ease;
    }
    .req-item.checked .check-icon {
        color: #0f766e;
    }
  </style>
</head>
<body class="bg-brand-stone text-slate-800 font-sans antialiased leading-relaxed">

  <!-- Navigation -->
  <nav class="fixed w-full bg-white/90 backdrop-blur-md shadow-sm z-50 border-b border-stone-
200">
```

```
    <div class="max-w-6xl mx-auto px-6 py-4 flex justify-between items-center">
      <div class="text-xl font-bold text-brand-slate tracking-tight">
        <span class="text-brand-teal">HLF</span> Explorer
      </div>
      <div class="hidden md:flex space-x-8 text-sm font-medium text-slate-600">
        <a href="#objective" class="hover:text-brand-teal transition">Objective</a>
        <a href="#requirements" class="hover:text-brand-teal transition">Requirements</a>
        <a href="#comparison" class="hover:text-brand-teal transition">Comparison</a>
        <a href="#architecture" class="hover:text-brand-teal transition">Architecture</a>
      </div>
    </div>
  </nav>

  <!-- Hero / Objective Section -->
  <section id="objective" class="pt-32 pb-20 px-6">
    <div class="max-w-4xl mx-auto text-center">
      <div class="inline-block bg-brand-accent text-brand-teal px-3 py-1 rounded-full text-xs font-semibold mb-6 tracking-wide uppercase">Research Brief</div>
      <h1 class="text-4xl md:text-5xl font-extrabold text-brand-slate mb-6 leading-tight">
        Hyperledger Fabric vs. Ethereum
      </h1>
      <p class="text-lg text-slate-600 mb-8 max-w-2xl mx-auto">
        A practical exploration objective: To study and deploy a basic Fabric test network with simple chaincode, and comprehensively compare its architecture, consensus, privacy, and performance against public blockchains like Ethereum.
      </p>
      <a href="#comparison" class="inline-block bg-brand-slate text-white px-8 py-3 rounded-lg font-medium hover:bg-slate-700 transition shadow-lg shadow-slate-200">
        Explore the Comparison
      </a>
    </div>
  </section>
```

```
<!-- Requirements Section -->
<section id="requirements" class="py-16 bg-white border-y border-stone-200">
  <div class="max-w-6xl mx-auto px-6">
    <div class="mb-12 max-w-2xl">
      <h2 class="text-3xl font-bold text-brand-slate mb-4">Project Prerequisites</h2>
      <p class="text-slate-600">
        This section outlines the necessary environment setup to execute the project. Interact with the lists below to check off your completed setup steps.
      </p>
    </div>

    <div class="grid md:grid-cols-2 gap-12">
      <!-- Software -->
      <div class="bg-stone-50 rounded-xl p-8 border border-stone-100 shadow-sm">
        <div class="flex items-center mb-6">
          <span class="text-2xl mr-3 text-brand-teal">&#128187;</span>
          <h3 class="text-xl font-bold text-brand-slate">Software Stack</h3>
        </div>
        <ul class="space-y-4" id="software-list">
          <!-- Populated by JS -->
        </ul>
      </div>

      <!-- Hardware -->
      <div class="bg-stone-50 rounded-xl p-8 border border-stone-100 shadow-sm">
        <div class="flex items-center mb-6">
          <span class="text-2xl mr-3 text-brand-teal">&#9881;</span>
          <h3 class="text-xl font-bold text-brand-slate">Hardware & Network</h3>
        </div>
        <ul class="space-y-4" id="hardware-list">
```

```
                <!-- Populated by JS -->
              </ul>
            </div>
          </div>
        </div>
      </section>

      <!-- Comparison Section -->
      <section id="comparison" class="py-20">
        <div class="max-w-6xl mx-auto px-6">
          <div class="text-center mb-16">
            <h2 class="text-3xl font-bold text-brand-slate mb-4">Comparative Analysis</h2>
            <p class="text-slate-600 max-w-2xl mx-auto">
              Analyzing the fundamental differences between permissioned (Hyperledger Fabric) and permissionless (Ethereum) blockchain architectures across key performance indicators.
            </p>
          </div>

          <div class="grid lg:grid-cols-2 gap-12 items-start">

            <!-- Radar Chart Container -->
            <div class="bg-white p-6 rounded-2xl shadow-sm border border-stone-200 flex flex-col justify-center items-center">
              <h3 class="text-lg font-semibold text-center text-slate-700 mb-2">Metrics Overview</h3>
              <p class="text-xs text-center text-slate-500 mb-6">(Scale 1-10: Higher represents greater capability/intensity in that metric)</p>
              <div class="chart-container">
                <canvas id="comparisonChart"></canvas>
              </div>
            </div>

            <!-- Interactive Tabs -->
```

```
<div class="bg-white rounded-2xl shadow-sm border border-stone-200 overflow-hidden">
  <div class="flex border-b border-stone-200 bg-stone-50">
    <button class="tab-btn flex-1 py-4 text-sm font-semibold text-brand-teal border-b-2 border-brand-teal bg-white" data-target="arch">Architecture</button>
    <button class="tab-btn flex-1 py-4 text-sm font-medium text-slate-500 hover:text-slate-700 hover:bg-stone-100 transition" data-target="consensus">Consensus</button>
    <button class="tab-btn flex-1 py-4 text-sm font-medium text-slate-500 hover:text-slate-700 hover:bg-stone-100 transition" data-target="privacy">Privacy</button>
    <button class="tab-btn flex-1 py-4 text-sm font-medium text-slate-500 hover:text-slate-700 hover:bg-stone-100 transition" data-target="usecase">Use Cases</button>
  </div>

  <div class="p-8 h-full min-h-[350px]">
    <!-- Tab Content 1 -->
    <div id="arch" class="tab-content block">
      <h4 class="text-xl font-bold text-brand-slate mb-4">Architecture</h4>
      <div class="space-y-4">
        <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-brand-teal">
          <strong class="block text-brand-slate mb-1">Hyperledger Fabric</strong>
          <p class="text-sm text-slate-600">Permissioned and private network. Employs a modular architecture with pluggable components (consensus, membership services). Nodes have distinct roles (Endorsers, Committers, Orderers).</p>
        </div>
        <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-blue-500">
          <strong class="block text-brand-slate mb-1">Ethereum</strong>
          <p class="text-sm text-slate-600">Public and permissionless (by default). Flat architecture where all full nodes maintain the entire state ledger and participate equally in transaction validation and smart contract execution.</p>
        </div>
      </div>
    </div>

    <!-- Tab Content 2 -->
    <div id="consensus" class="tab-content hidden">
```

```
<h4 class="text-xl font-bold text-brand-slate mb-4">Consensus Mechanism</h4>
<div class="space-y-4">
  <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-brand-teal">
    <strong class="block text-brand-slate mb-1">Hyperledger Fabric</strong>
    <p class="text-sm text-slate-600">Pluggable consensus. Typically uses Crash Fault Tolerant (CFT) mechanisms like Raft. Separates transaction endorsement from ordering, allowing for higher throughput and scalability.</p>
  </div>
  <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-blue-500">
    <strong class="block text-brand-slate mb-1">Ethereum</strong>
    <p class="text-sm text-slate-600">Transitioned to Proof of Stake (PoS). Emphasizes Byzantine Fault Tolerance (BFT) in an untrusted environment. Slower transaction finality compared to CFT models but higher decentralization.</p>
  </div>
</div>
</div>

<!-- Tab Content 3 -->
<div id="privacy" class="tab-content hidden">
  <h4 class="text-xl font-bold text-brand-slate mb-4">Privacy & Confidentiality</h4>
  <div class="space-y-4">
    <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-brand-teal">
      <strong class="block text-brand-slate mb-1">Hyperledger Fabric</strong>
      <p class="text-sm text-slate-600">High privacy. Utilizes "Channels" for restricted data sharing between specific consortium members, and "Private Data Collections" for isolating sensitive transaction details even within a channel.</p>
    </div>
    <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-blue-500">
      <strong class="block text-brand-slate mb-1">Ethereum</strong>
      <p class="text-sm text-slate-600">Low native privacy. All transactions and smart contract states are publicly visible on the ledger. Requires complex Layer 2 solutions (like ZK-Rollups) for transactional privacy.</p>
    </div>
  </div>
```

```
                </div>


                <!-- Tab Content 4 -->
                <div id="usecase" class="tab-content hidden">
                  <h4 class="text-xl font-bold text-brand-slate mb-4">Target Use Cases</h4>
                  <div class="space-y-4">
                    <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-brand-teal">
                      <strong class="block text-brand-slate mb-1">Hyperledger Fabric</strong>
                      <p class="text-sm text-slate-600">B2B (Business-to-Business) applications.
Supply chain tracking, financial trade settlements, healthcare data management, identity networks.
Environments requiring strict regulatory compliance.</p>
                    </div>
                    <div class="p-4 rounded-lg bg-stone-50 border-l-4 border-blue-500">
                      <strong class="block text-brand-slate mb-1">Ethereum</strong>
                      <p class="text-sm text-slate-600">B2C (Business-to-Consumer) and DeFi
(Decentralized Finance). Public token issuance (ICOs/NFTs), decentralized exchanges, public DAOs,
and global, censorship-resistant applications.</p>
                    </div>
                  </div>
                </div>
              </div>
            </div>
          </div>
        </div>
      </section>


      <!-- Architecture / Implementation Flow -->
      <section id="architecture" class="py-20 bg-brand-slate text-white">
        <div class="max-w-6xl mx-auto px-6">
          <div class="text-center mb-16">
            <h2 class="text-3xl font-bold mb-4">Fabric Test Network Architecture</h2>
            <p class="text-slate-400 max-w-2xl mx-auto">
```

A conceptual overview of deploying a simple chaincode. Fabric utilizes a unique "Execute-Order-Validate" architecture, represented in the logical flow below.

</p>

<!-- CSS/HTML diagram replacing Mermaid/SVG -->

<!-- Client -->

📱

Client App

Initiates Transaction

<!-- Tooltip -->

Proposes transaction via SDK.

➞

<!-- Peer -->

▦

Endorsing Peer

Executes Chaincode

<!-- Tooltip -->

```
<div class="absolute bottom-full left-1/2 transform -translate-x-1/2 mb-2 w-48 p-2 bg-black text-xs rounded opacity-0 group-hover:opacity-100 transition pointer-events-none z-10">
Simulates transaction, signs result, returns endorsement.
</div>
</div>

<div class="text-2xl text-slate-500 rotate-90 md:rotate-0">&#10142;</div>

<!-- Orderer -->
<div class="bg-slate-700 p-6 rounded-xl text-center w-48 border border-slate-600 border-t-4 border-t-yellow-500 relative group cursor-pointer hover:bg-slate-600 transition">
<div class="text-3xl mb-2 text-yellow-500">&#9776;</div>
<div class="font-bold">Ordering Node</div>
<div class="text-xs text-slate-400 mt-2">Packages Blocks</div>

<!-- Tooltip -->
<div class="absolute bottom-full left-1/2 transform -translate-x-1/2 mb-2 w-48 p-2 bg-black text-xs rounded opacity-0 group-hover:opacity-100 transition pointer-events-none z-10">
Orders endorsed transactions into blocks chronologically.
</div>
</div>

<div class="text-2xl text-slate-500 rotate-90 md:rotate-0">&#10142;</div>

<!-- Ledger -->
<div class="bg-slate-700 p-6 rounded-xl text-center w-48 border border-slate-600 border-t-4 border-t-blue-400 relative group cursor-pointer hover:bg-slate-600 transition">
<div class="text-3xl mb-2 text-blue-400">&#128218;</div>
<div class="font-bold">Committing Peer</div>
<div class="text-xs text-slate-400 mt-2">Updates Ledger</div>

<!-- Tooltip -->
```

```
                <div class="absolute bottom-full left-1/2 transform -translate-x-1/2 mb-2 w-48 p-2 bg-black text-xs rounded opacity-0 group-hover:opacity-100 transition pointer-events-none z-10">
                    Validates block, appends to chain, updates state database.
                </div>
            </div>

        </div>

        <div class="mt-12 text-center text-sm text-slate-400 max-w-xl mx-auto bg-slate-800 p-4 rounded-lg">
            <span class="text-brand-accent font-bold">Note on Chaincode:</span> In Fabric, chaincode (smart contracts) can be written in Node.js or Go, installed on peers, and instantiated on a specific channel.
        </div>
    </div>
  </section>

  <!-- Footer -->
  <footer class="bg-brand-slate text-slate-500 py-8 text-center border-t border-slate-700 text-sm">
    <p>Created for "Hyperledger Fabric – Comparative Exploration" Study.</p>
  </footer>

  <!-- Interactive Logic -->
  <script>
    document.addEventListener('DOMContentLoaded', () => {

      // --- 1. Populate and Handle Requirements Checklist ---
      const softwareReqs = [
        "Ubuntu 20.04 / Windows (WSL) / macOS",
        "Docker & Docker Compose",
        "Git installed",
        "cURL command line tool",
```

```
    "Node.js (v16+) or Go language",
    "Hyperledger Fabric binaries & samples downloaded"
  ];

  const hardwareReqs = [
    "Minimum 8 GB RAM available",
    "Stable Internet connection"
  ];

  const createListItems = (items, containerId) => {
    const container = document.getElementById(containerId);
    items.forEach((item, index) => {
      const li = document.createElement('li');
      li.className = 'req-item flex items-start cursor-pointer group p-2 rounded hover:bg-white
transition';
      li.innerHTML = `
        <div class="mt-0.5 mr-3 w-5 h-5 rounded border border-slate-300 flex items-center
justify-center bg-white group-hover:border-brand-teal transition">
          <span class="check-icon opacity-0 text-sm">&#10004;</span>
        </div>
        <span class="text-slate-700 font-medium select-none text-sm">${item}</span>
      `;

      // Toggle functionality
      li.addEventListener('click', function() {
        this.classList.toggle('checked');
        const icon = this.querySelector('.check-icon');
        if(this.classList.contains('checked')) {
          icon.style.opacity = '1';
        } else {
          icon.style.opacity = '0';
        }
```

```
        });

        container.appendChild(li);
    });
};

createListItems(softwareReqs, 'software-list');
createListItems(hardwareReqs, 'hardware-list');

// --- 2. Chart.js Radar Chart (Comparison) ---
const ctx = document.getElementById('comparisonChart').getContext('2d');

Chart.defaults.font.family = "'Inter', sans-serif";
Chart.defaults.color = '#64748b';

const comparisonChart = new Chart(ctx, {
    type: 'radar',
    data: {
        labels: [
            'Transaction Throughput',
            'Privacy/Confidentiality',
            'Decentralization (Nodes)',
            'Energy Efficiency',
            'Setup Complexity'
        ],
        datasets: [{
            label: 'Hyperledger Fabric',
            data: [9, 9, 4, 9, 8],
            backgroundColor: 'rgba(15, 118, 110, 0.2)', // brand-teal with opacity
            borderColor: 'rgba(15, 118, 110, 1)',
            pointBackgroundColor: 'rgba(15, 118, 110, 1)',
```

```
        pointBorderColor: '#fff',
        pointHoverBackgroundColor: '#fff',
        pointHoverBorderColor: 'rgba(15, 118, 110, 1)',
        borderWidth: 2
    }, {
        label: 'Ethereum (Public)',
        data: [4, 2, 10, 8, 3], // Note: Eth energy efficiency improved post-merge to PoS
        backgroundColor: 'rgba(59, 130, 246, 0.2)', // blue-500 with opacity
        borderColor: 'rgba(59, 130, 246, 1)',
        pointBackgroundColor: 'rgba(59, 130, 246, 1)',
        pointBorderColor: '#fff',
        pointHoverBackgroundColor: '#fff',
        pointHoverBorderColor: 'rgba(59, 130, 246, 1)',
        borderWidth: 2
    }]
},
options: {
    responsive: true,
    maintainAspectRatio: false,
    scales: {
        r: {
            angleLines: { color: 'rgba(0, 0, 0, 0.05)' },
            grid: { color: 'rgba(0, 0, 0, 0.05)' },
            pointLabels: {
                font: { size: 11, weight: '600' },
                color: '#475569'
            },
            ticks: {
                beginAtZero: true,
                max: 10,
                stepSize: 2,
```

```
                display: false // Hide numbers on axis
            }
        }
    },
    plugins: {
        legend: {
            position: 'bottom',
            labels: {
                padding: 20,
                font: { weight: 'bold' }
            }
        },
        tooltip: {
            backgroundColor: '#1e293b',
            titleFont: { size: 13 },
            bodyFont: { size: 13 },
            padding: 10,
            cornerRadius: 8,
            callbacks: {
                label: function(context) {
                    return ` ${context.dataset.label}: ${context.raw}/10`;
                }
            }
        }
    }
  }
});

// --- 3. Interactive Tabs Logic ---
const tabBtns = document.querySelectorAll('.tab-btn');
const tabContents = document.querySelectorAll('.tab-content');
```

```
        tabBtns.forEach(btn => {
          btn.addEventListener('click', () => {
            // Remove active states from all
            tabBtns.forEach(b => {
              b.classList.remove('text-brand-teal', 'border-b-2', 'border-brand-teal', 'bg-white');
              b.classList.add('text-slate-500', 'hover:text-slate-700', 'hover:bg-stone-100');
            });
            tabContents.forEach(c => {
              c.classList.remove('block');
              c.classList.add('hidden');
            });

            // Add active state to clicked
            btn.classList.remove('text-slate-500', 'hover:text-slate-700', 'hover:bg-stone-100');
            btn.classList.add('text-brand-teal', 'border-b-2', 'border-brand-teal', 'bg-white');

            const targetId = btn.getAttribute('data-target');
            document.getElementById(targetId).classList.remove('hidden');
            document.getElementById(targetId).classList.add('block');
          });
        });

      });
    </script>
</body>
</html>
```

OUTPUT:

RESEARCH BRIEF

# Hyperledger Fabric vs. Ethereum

A practical exploration objective: To study and deploy a basic Fabric test network with simple chaincode, and comprehensively compare its architecture, consensus, privacy, and performance against public blockchains like Ethereum.

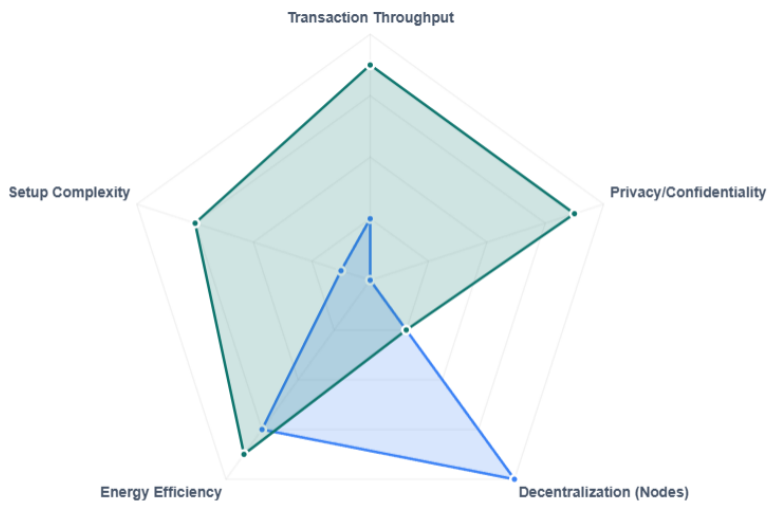
Explore the Comparison

HLF Explorer

Objective Requirements Comparison Architecture

## Metrics Overview

(Scale 1-10: Higher represents greater capability/intensity in that metric)



HLF Explorer
Objective
Requirements
Comparison
Architecture
Fabric Test Network Architecture
A conceptual overview of deploying a simple chaincode. Fabric utilizes a unique "Execute-Order-Validate" architecture, represented in the logical flow below.
Client App
Initiates Transaction
Endorsing Peer
Executes Chaincode
Ordering Node
Packages Blocks
Committing Peer
Updates Ledger